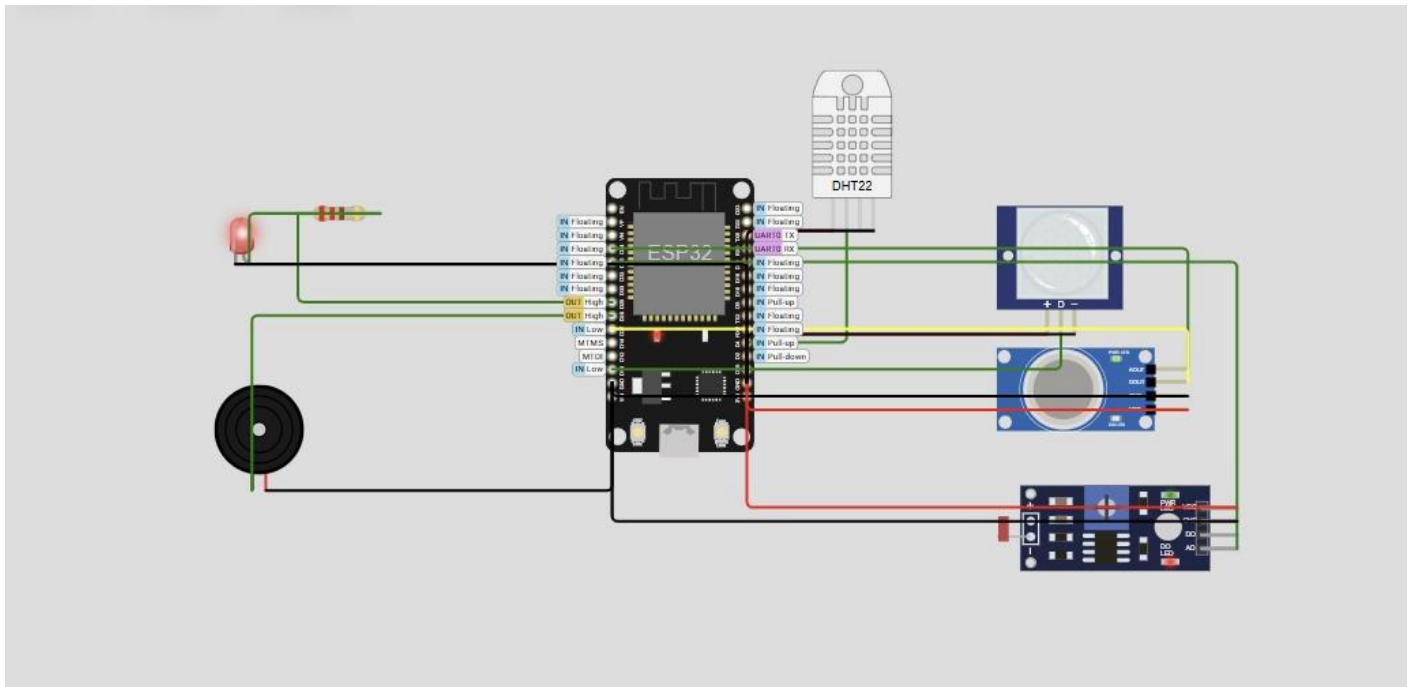


IOT MONITORING PLATFORM

Surveillance environnementale intelligente

ESP32 • MQTT • Elasticsearch • Kibana • FastAPI • Docker Compose



Rapport final de projet académique
Année universitaire 2025-2026

Résumé exécutif

Ce projet met en œuvre une plateforme complète de surveillance environnementale IoT. Un ESP32 simulé dans Wokwi collecte la température, l'humidité, le mouvement, le niveau de gaz et la luminosité. Les mesures sont publiées toutes les cinq secondes via MQTT, traitées par un consumer Python, contrôlées par des seuils d'anomalie puis stockées dans Elasticsearch. FastAPI expose les données en lecture seule et Kibana permet leur exploration en temps réel. L'ensemble des services applicatifs est orchestré avec Docker Compose.

1. Contexte et problématique

La surveillance manuelle ou fragmentée ne permet pas de réagir rapidement à une surchauffe, à une humidité excessive ou à une concentration élevée de gaz. L'objectif est de transformer des mesures brutes en informations horodatées, historisées et immédiatement exploitables.

- Acquérir plusieurs signaux environnementaux sur un même nœud.
- Transporter les données avec un protocole léger et découplé.
- Détecter automatiquement les dépassements de seuil.
- Déclencher une alerte locale et une notification distante.
- Visualiser l'état courant et l'historique.

2. Objectifs fonctionnels

Objectif	Résultat attendu
Acquisition	Lecture de quatre capteurs simulés
Publication	Message JSON toutes les 5 secondes sur <code>iot/sensors</code>
Détection	Température > 35 °C, humidité > 80 %, gaz > 1000 ppm
Alerting	LED, buzzer et webhook Slack/Discord/Teams
Stockage	Document indexé dans Elasticsearch
Consultation	API FastAPI et tableaux de bord Kibana

3. Architecture globale

Le flux principal suit la chaîne suivante : ESP32/Wokwi → broker MQTT public → consumer Python → Elasticsearch. Les flux d'exploitation sont assurés par FastAPI et Kibana, tandis que le webhook porte les alertes sortantes.

Architecture logique :

Capteurs → ESP32 → broker.hivemq.com → topic `iot/sensors` → consumer Python → Elasticsearch → FastAPI / Kibana

4. Nœud capteur ESP32

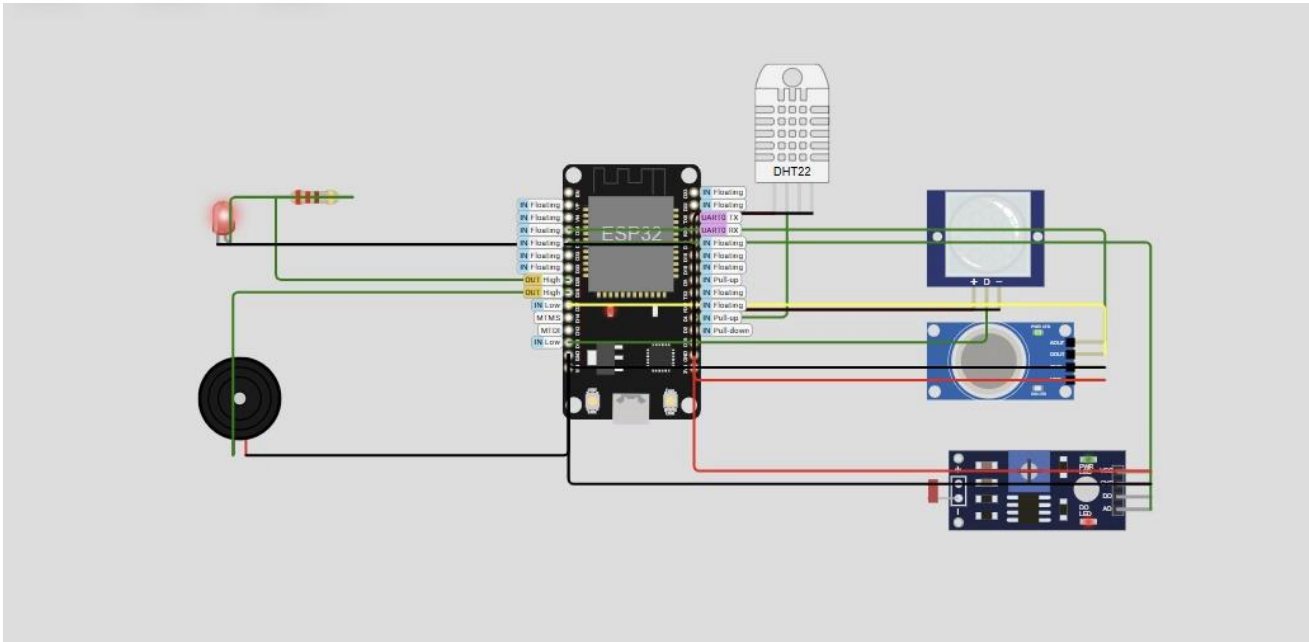


Figure 1. Circuit complet : ESP32, DHT22, PIR, MQ2, LDR, LED et buzzer.

Composant	Fonction
DHT22	Température et humidité
PIR	Détection de mouvement
MQ2	Simulation du niveau de gaz
LDR	Mesure de luminosité
LED + buzzer	Alerte locale immédiate
ESP32	Acquisition, logique locale et publication MQTT

5. Simulation Wokwi

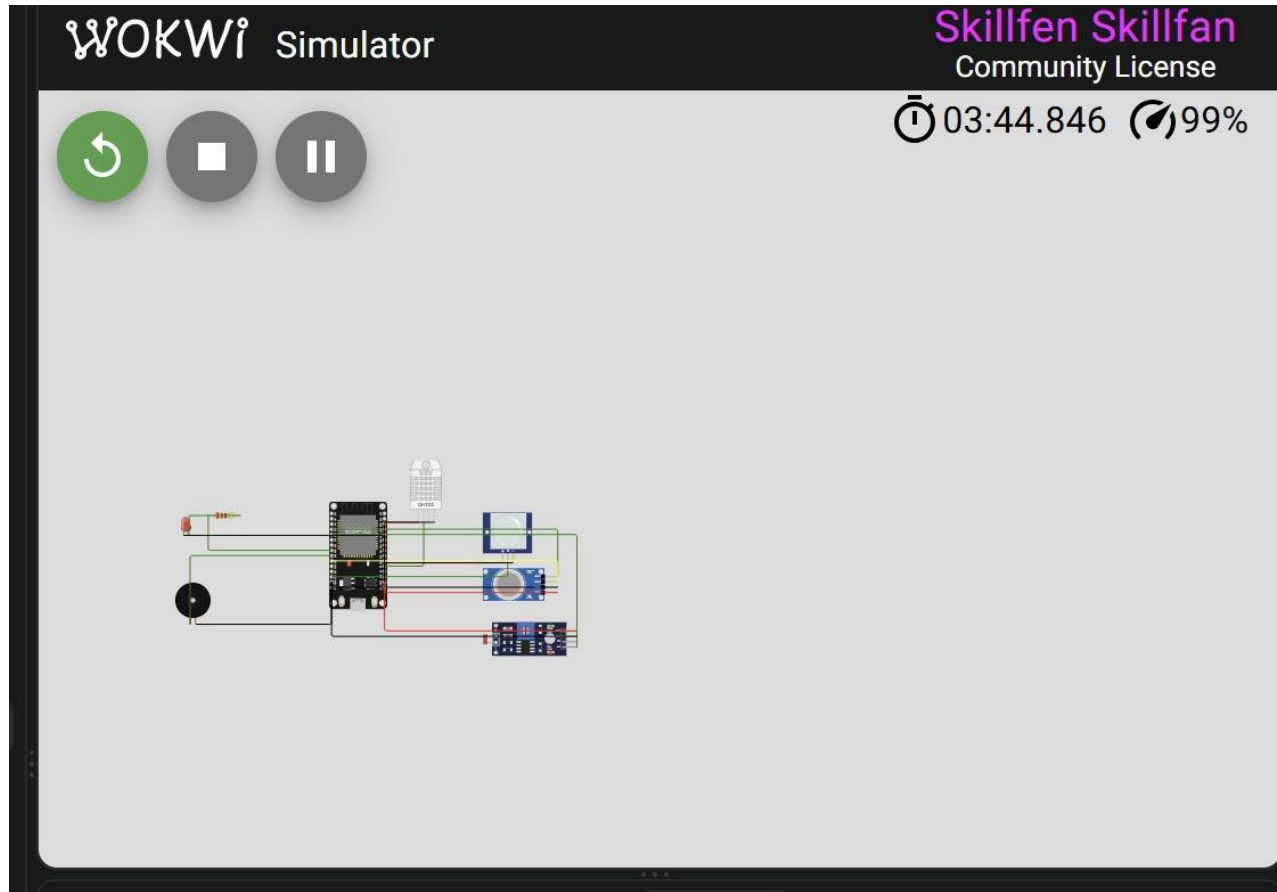


Figure 2. Simulation Wokwi en cours d'exécution.

Wokwi permet de démontrer le fonctionnement sans matériel physique. Les valeurs peuvent être modifiées pour provoquer volontairement un scénario normal ou une anomalie. L'ESP32 se connecte au réseau Wokwi-GUEST puis au broker public HiveMQ.

6. Pipeline MQTT et format des données

Le broker utilisé est broker.hivemq.com sur le port 1883. L'ESP32 publie sur le topic `iot/sensors`. Le modèle publish-subscribe découple le capteur du stockage et permet de remplacer ou multiplier les consommateurs sans modifier le firmware.

Exemple de message JSON :

```
{
  "device_id": "esp32-01",
  "temperature": 24.0,
  "humidity": 40.0,
  "motion": false,
  "gas_ppm": 8859.0,
  "light": 735
}
```

7. Consumer Python et détection d'anomalies

```
2026-09-08 17:42:19,040 [INFO] POST http://localhost:9200/iot-data/_doc [status:201 duration:0.063s]
2026-09-08 17:42:19,041 [INFO] Stored reading | device=esp32-01 temp=24.0 humidity=40.0 motion=False gas_ppm=8859.6 anomaly=True
```

Figure 3. Le consumer stocke une mesure et détecte une anomalie de gaz.

La capture confirme une requête POST vers Elasticsearch avec le statut HTTP 201. La mesure issue du device esp32-01 contient temperature=24.0, humidity=40.0, motion=False et gas_ppm=8859.0. Le champ anomaly=True est cohérent avec le seuil gaz fixé à 1000 ppm.

- Validation du JSON reçu.
- Ajout d'un horodatage côté serveur.
- Calcul de l'état d'anomalie.
- Application d'un cooldown pour limiter les alertes répétées.
- Envoi éventuel d'un webhook puis indexation.

8. Stockage Elasticsearch

Chaque mesure devient un document indexé. Un mapping explicite facilite les agrégations temporelles, les filtres par device et le comptage d'anomalies.

Champ	Type logique	Utilisation
@timestamp	date	Analyse temporelle
device_id	keyword	Filtrage par équipement
temperature / humidity	float	Courbes et seuils
gas_ppm / light	float	Analyse environnementale
motion	boolean	Événement de présence
anomaly	boolean	Filtre et compteur
anomaly_types	keyword	Qualification de l'incident

9. API REST FastAPI

IoT Monitoring API 1.0.0 OAS 3.1
/openapi.json

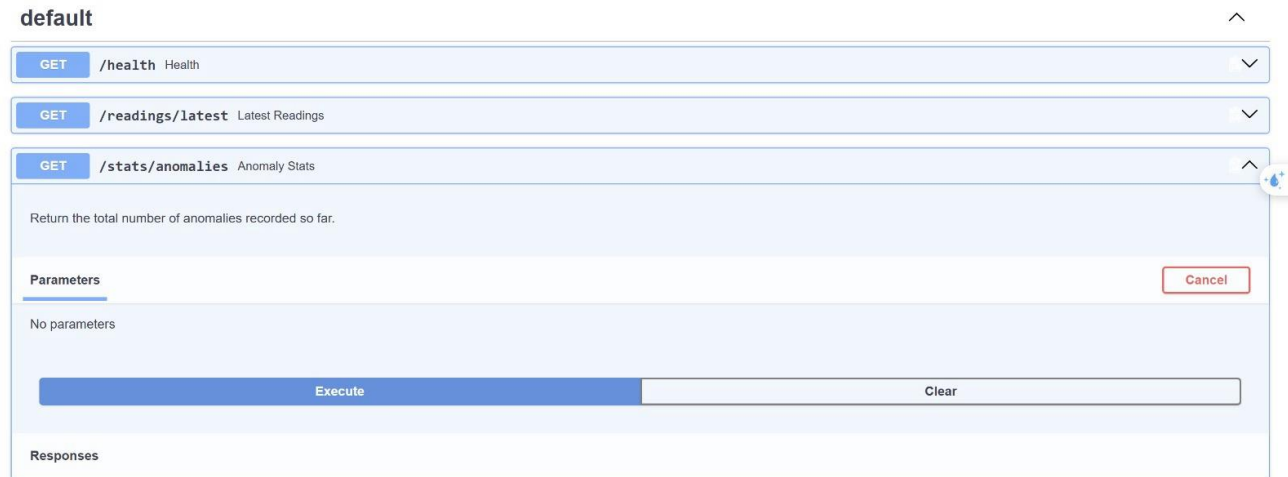


Figure 4. Documentation Swagger de l'API IoT Monitoring.

L'API est volontairement en lecture seule. La documentation interactive Swagger présente notamment /health, /readings/latest et /stats/anomalies.

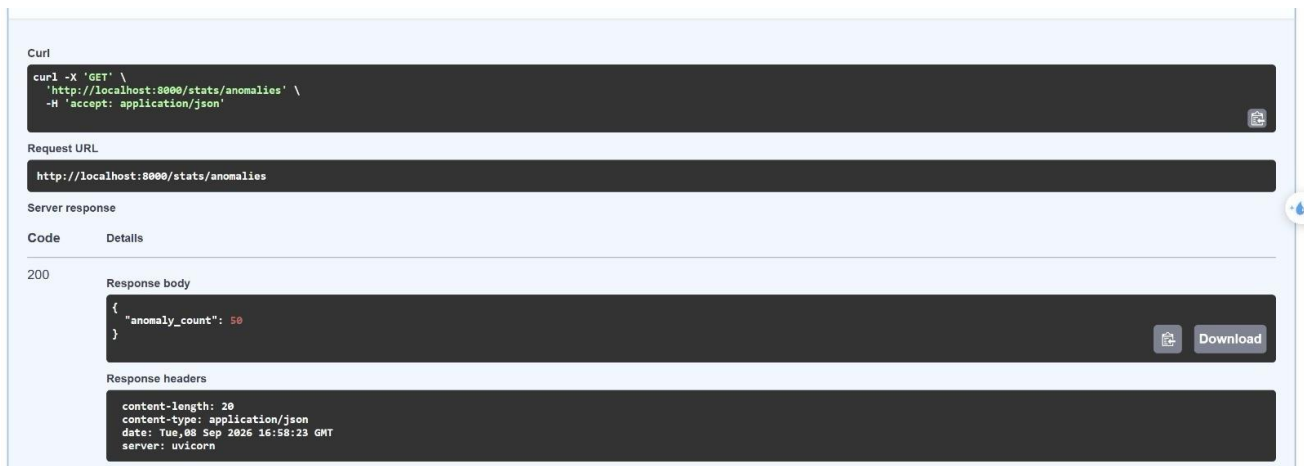


Figure 5. Réponse HTTP 200 de l'endpoint /stats/anomalies.

La réponse observée contient anomaly_count = 50. Cette preuve confirme la communication entre FastAPI et Elasticsearch ainsi que l'agrégation des documents marqués comme anomalies.

10. Conteneurisation Docker Compose

CREATED	STATUS	PORTS	NAME
S			
f4e100f99a9a	iot-monitoring-api	"uvicorn app.main:ap...	
" 2 hours ago	Up 2 hours	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp	iot-
api			
85745fcb5ad3	docker.elastic.co/kibana/kibana:8.15.0	"/bin/tini -- /usr/l...	
" 2 hours ago	Up 2 hours	0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp	iot-
kibana			
e989dd2b59fc	docker.elastic.co/elasticsearch/elasticsearch:8.15.0	"/bin/tini -- /usr/l...	
" 2 hours ago	Up 2 hours (healthy)	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp	iot-
elasticsearch			

Figure 6. Services Docker actifs : API, Kibana et Elasticsearch.

La capture montre trois services démarrés depuis deux heures : iot-monitoring-api sur le port 8000, Kibana 8.15.0 sur le port 5601 et Elasticsearch 8.15.0 sur le port 9200. Elasticsearch apparaît en état healthy. Le consumer peut être exécuté comme service supplémentaire ou localement selon la configuration retenue.

11. Visualisation Kibana

Kibana exploite les documents Elasticsearch à travers un Data View basé sur iot-data-*. Le tableau de bord recommandé contient :

- Indicateurs de dernière température, humidité et concentration de gaz.
- Courbes temporelles de température et d'humidité.
- Nombre total d'anomalies et répartition par type.
- Table des alertes récentes avec device, timestamp et valeurs.
- Filtres par device et période.

12. Scénario de démonstration

Étape	Action	Preuve
1	Démarrer la stack	<code>docker compose up -d</code>
2	Lancer le consumer	Connexion au broker et abonnement au topic
3	Démarrer Wokwi	Publication toutes les 5 secondes
4	Créer une anomalie	Passer le gaz au-dessus de 1000 ppm
5	Vérifier le terminal	<code>anomaly=True</code> et status 201
6	Interroger l'API	<code>/stats/anomalies</code> retourne HTTP 200
7	Ouvrir Kibana	Afficher la mesure et l'historique

13. Résultats et validation

- Le circuit simulé fonctionne avec les quatre familles de capteurs.
- La publication MQTT alimente le consumer en continu.
- Une anomalie de gaz à 8859 ppm a été identifiée.
- La mesure a été stockée dans Elasticsearch avec un statut 201.
- Elasticsearch est healthy et les ports applicatifs sont exposés.
- FastAPI retourne un statut 200 et un compteur de 50 anomalies.

14. Limites et perspectives

- Remplacer le broker public par un broker privé authentifié.
- Activer TLS et gérer les secrets hors du code source.
- Ajouter une file de reprise, des retries et une dead-letter queue.
- Utiliser une détection statistique ou un modèle de machine learning.
- Ajouter l'identification de plusieurs devices et la gestion de flotte.
- Intégrer des tests automatisés du firmware, du consumer et de l'API.

Conclusion

La plateforme réalisée démontre une chaîne IoT cohérente de bout en bout : acquisition, communication, traitement, détection, stockage, API et visualisation. Le recours à Wokwi et Docker Compose rend la démonstration reproductible. Les résultats observés, notamment l'indexation HTTP 201, l'anomalie de gaz et la réponse FastAPI HTTP 200, confirment le fonctionnement des composants principaux. Le projet constitue une base solide pour une plateforme de surveillance plus sécurisée, multi-capteurs et multi-devices.

Annexe A. Commandes principales

```
docker compose up -d
docker compose ps
pip install -r requirements.txt
python mqtt_consumer.py
http://localhost:8000/docs
http://localhost:5601
http://localhost:9200
```